PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

GROUP BY

Сергей Соловьев, Тантор Лабс

```
SELECT max(age) FROM users;
```

```
SELECT userid, avg(transferred) FROM user_transactions GROUP BY userid;
```

```
state = init()

for tuple in input:
  state = transit(state, tuple)

result = finalize(state)
```

```
select aggfnoid, agginitval, aggtransfn, aggfinalfn from pg_aggregate;

        aggfnoid        |    agginitval    |    aggtransfn    |      aggfinalfn
------------------------+------------------+------------------+---------------------
pg_catalog.stddev       | {0,0,0}          | float4_accum     | float8_stddev_samp
pg_catalog.avg          | {0,0,0}          | float4_accum     | float8_avg
pg_catalog.avg          | {0,0}            | int4_avg_accum   | int8_avg
pg_catalog.max          |                  | int4larger       | -
pg_catalog.max          |                  | float4larger     | -
```

Aggregate

```
EXPLAIN SELECT avg(a) FROM tbl;

            QUERY PLAN
-------------------------------------
Aggregate
  ->  Seq Scan on tbl
(2 rows)
```

```
static TupleTableSlot *ExecAgg(PlanState *pstate)
{
    AggState *node = castNode(AggState, pstate);
    switch (node->aggstrategy)
    {
        case AGG_HASHED:
            /* ... */
        case AGG_MIXED:
            /* ... */
        case AGG_SORTED:
        case AGG_PLAIN:
            return agg_retrieve_direct(node);
    }
}
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static void initialize_aggregates(AggState *aggstate,
                                  AggStatePerGroup *pergroups)
{
    int numTrans = aggstate->numtrans;
    AggStatePerTrans transstates = aggstate->pertrans;

    for (int transno = 0; transno < numTrans; transno++)
    {
        AggStatePerTrans pertrans = &transstates[transno];
        AggStatePerGroup pergroupstate = &pergroup[transno];
        initialize_aggregate(aggstate, pertrans, pergroupstate);
    }
}
```

```
struct AggStatePerGroup
{
    Datum transValue;
    bool  transValueIsNull;
    bool  noTransValue;
}
```

```
state = (0, 0)

for tuple in input:
  state = int4_agg_accum(state, tuple)

result = int8_avg(state)
```

```
struct AggStatePerTrans
{
   Datum initValue;
   Oid   transfn_oid;
}
```

```
state = (0, 0)

for tuple in input:
  state = int4_agg_accum(state, tuple)

result = int8_avg(state)
```

```
struct AggStatePerAgg
{
   Oid finalfn_oid;
}
```

```
select agginitval, aggtransfn, count(*) from pg_aggregate
  group by 1, 2 having count(*) > 1
  order by 3 desc;

 agginitval   |          aggtransfn          | count
--------------+------------------------------+-----
{0,0,0,0,0,0} | float8_regr_accum            |  11
{0,0,0}       | float4_accum                 |   7
              | ordered_set_transition       |   7
{0,0,0}       | float8_accum                 |   7
              | int4_accum                   |   6
              | numeric_accum                |   6
              | int2_accum                   |   6
... omitted
```

```
select avg(a::float), stddev(a::float) from tbl;
```

PG VARIABLES

VARIABLES

Locals

node: AggState * = 0x56819c8cf678

ss

aggs: List * = 0x56819c8d0770

numaggs: int = 2

numtrans: int = 1

aggstrategy: AggStrategy = AGG_PL...

aggsplit: AggSplit = AGGSPLIT_SIM...

phase: AggStatePerPhase = 0x56819...

WATCH

CALL STACK Paused on breakpoint

src > backend > executor > nodeAgg.c > ExecAg

```
2244  ExecAgg(PlanState *pstat
2251      if (!node->agg_done)
2254          switch (node->ph
2261              result =
2262              break;
2263          case AGG_PLA
2264          case AGG_SOR
2265              result =
2266              break;
2267      }
2268
```

```
static void initialize_aggregate(AggState *aggstate, AggStatePerTrans pertrans,
                                 AggStatePerGroup pergroupstate)
{
    if (pertrans->initValueIsNull)
        pergroupstate->transValue = pertrans->initValue;
    else
    {
        pergroupstate->transValue = datumCopy(pertrans->initValue,
                                              pertrans->transtypeByVal,
                                              pertrans->transtypeLen);
    }
    pergroupstate->transValueIsNull = pertrans->initValueIsNull;
    pergroupstate->noTransValue = pertrans->initValueIsNull;
}
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static Datum ExecInterpExpr(ExprState *state, ExprContext *econtext, bool *isnull)
{
    EEO_SWITCH()
    {
        EEO_CASE(EEOP_OUTER_FETCHSOME)
        {
            slot_getsomeattrs(outerslot, op->d.fetch.last_var);
            EEO_NEXT();
        }
        EEO_CASE(EEOP_AGG_PLAIN_TRANS_BYVAL)
        {
            AggState   *aggstate = castNode(AggState, state->parent);
            AggStatePerTrans pertrans = op->d.agg_trans.pertrans;
            AggStatePerGroup pergroup = &aggstate->all_pergroups[op->d.agg_trans.transno];
            ExecAggPlainTransByVal(aggstate, pertrans, pergroup,
                                   op->d.agg_trans.aggcontext);
            EEO_NEXT();
        }
    }
}
```

VARIABLES

Locals

transarray: ArrayType * = 0x5681…

newval: int32 = 1

transdata: Int8TransTypeData * =

count: int64 = 0

sum: int64 = 0

__func__: const char [15] = [15]

fcinfo: FunctionCallInfo = 0x568…

Registers

WATCH

CALL STACK Paused on breakpoint

int4_avg_accum(FunctionCallInfo fc

ExecAggPlainTransByRef(AggState *

ExecInterpExpr(ExprState * state,

ExecInterpExprStillValid(ExprState

ExecEvalExprNoReturn(ExprState * s

ExecEvalExprNoReturnSwitchContext(

```
6804  int4_avg_accum(PG_FUNCTION_ARGS)
6821          ARR_SIZE(transarray) != ARR_OVER
6822          elog(ERROR, "expected 2-element
6823
6824      transdata = (Int8TransTypeData *) AR
6825      transdata->count++;
6826      transdata->sum += newval;
6827
6828      PG_RETURN_ARRAYTYPE_P(transarray);
6829  }
6830
6831  Datum
6832  int4_avg_combine(PG_FUNCTION_ARGS)
6833  {
6834      ArrayType  *transarray1;
6835      ArrayType  *transarray2;
6836      Int8TransTypeData *state1;
6837      Int8TransTypeData *state2;
6838
```

VARIABLES

Locals

transarray: ArrayType * = 0x5681…

newval: int32 = 1

transdata: Int8TransTypeData * =

count: int64 = 1

sum: int64 = 1

__func__: const char [15] = [15]

fcinfo: FunctionCallInfo = 0x568…

Registers

WATCH

CALL STACK Paused on breakpoint

int4_avg_accum(FunctionCallInfo fc

ExecAggPlainTransByRef(AggState *

ExecInterpExpr(ExprState * state,

ExecInterpExprStillValid(ExprState

ExecEvalExprNoReturn(ExprState * s

ExecEvalExprNoReturnSwitchContext(

```
6804  int4_avg_accum(PG_FUNCTION_ARGS)
6821          ARR_SIZE(transarray) != ARR_OVER
6822          elog(ERROR, "expected 2-element
6823
6824      transdata = (Int8TransTypeData *) AR
6825      transdata->count++;
6826      transdata->sum += newval;
6827
6828      PG_RETURN_ARRAYTYPE_P(transarray);
6829  }
6830
6831  Datum
6832  int4_avg_combine(PG_FUNCTION_ARGS)
6833  {
6834      ArrayType  *transarray1;
6835      ArrayType  *transarray2;
6836      Int8TransTypeData *state1;
6837      Int8TransTypeData *state2;
6838
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static void finalize_aggregate(AggState *aggstate,
                               AggStatePerAgg peragg,
                               AggStatePerGroup pergroupstate,
                               Datum *resultVal, bool *resultIsNull)
{
    Datum result;
    InitFunctionCallInfoData(*fcinfo, &peragg->finalfn,
                             numFinalArgs,
                             pertrans->aggCollation,
                             (Node *) aggstate, NULL);
    *resultVal = FunctionCallInvoke(fcinfo);
    *resultIsNull = fcinfo->isnull;
}
```

PG VARIABLES

VARIABLES

Locals

transarray: ArrayType * = 0x56819c…

transdata: Int8TransTypeData * =

count: int64 = 200009

sum: int64 = 10013634

countd: Datum = 95114676469760

sumd: Datum = 95114676469792

__func__: const char [9] = [9]

fcinfo: FunctionCallInfo = 0x7fff8…

WATCH

CALL STACK Paused on breakpoint

int8_avg(FunctionCallInfo fcinfo)

finalize_aggregate(AggState * aggsta

finalize_aggregates(AggState * aggst

agg_retrieve_direct(AggState * aggst

ExecAgg(PlanState * pstate) 2265:1

src > backend > utils > adt > numeric.c > int8_avg(PG_F

```
6919  int8_avg(PG_FUNCTION_ARGS)
6932      if (transdata->count =
6933          PG_RETURN_NULL();
6934
6935      countd = NumericGetDat
6936      sumd = NumericGetDatum
6937
6938      PG_RETURN_DATUM(Direct
6939  }
6940
6941  /*
6942   * SUM(int2) and SUM(int4)
6943   * final function for both
6944   */
6945  Datum
```

```
SELECT a, avg(b) FROM tbl GROUP BY a;
                QUERY PLAN
-----------------------------------
HashAggregate
  Group Key: a
  ->  Seq Scan on tbl
```

```
SELECT a, b, c FROM tbl GROUP BY a, b, c;
                QUERY PLAN
-----------------------------------------
Group
  Group Key: a, b, c
  ->  Sort
        Sort Key: a, b, c
        ->  Seq Scan on tbl
```

GroupAggregate

```
EXPLAIN SELECT a, b FROM tbl GROUP BY a, b;

          QUERY PLAN
------------------------------
Group
  Group Key: a, b
  ->  Sort
        Sort Key: a, b
        ->  Seq Scan on tbl
```

1	1	1	(1, 1, 1)
1	1	1	
1	1	2	(1, 1, 2)
1	2	1	(1, 2, 1)
1	2	1	
2	2	1	(2, 2, 1)

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	2

Финализация

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

Финализация

1	1	2

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

2	2	1

Финализация

1	2	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Финализация

2	2	1

```
static TupleTableSlot *
ExecAgg(PlanState *pstate)
{
    switch (node->aggstrategy)
    {
        case AGG_MIXED:
        case AGG_MIXED:
           /* ... */
           break;
        case AGG_PLAIN:
        case AGG_SORTED:
           return agg_retrieve_direct(node);
    }
}
```

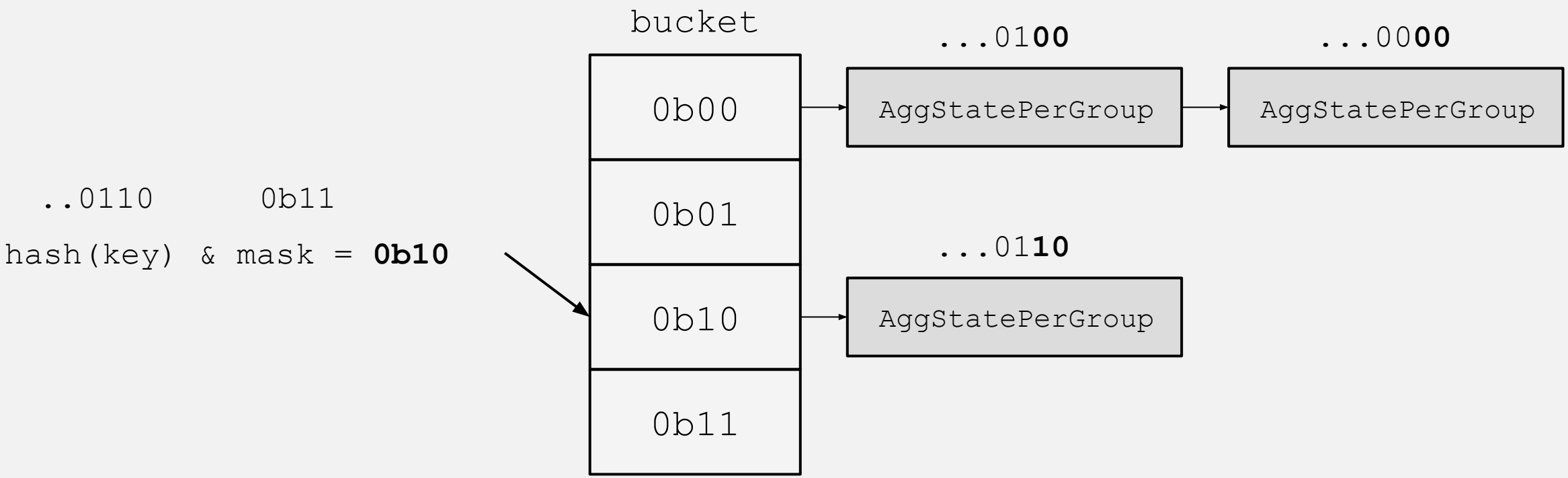
```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
    TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
    initialize_aggregates(aggstate, pergroups);
    for (;;)
    {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
            break;
        if (node->aggstrategy != AGG_PLAIN)
        {
            if (!ExecQual(aggstate->eqfunctions[node->numCols- 1], tmpcontext))
                break;
        }
    }

    finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
TypeCacheEntry *lookup_type_cache(Oid type_id, int flags)
{
    if (flags & TYPECACHE_EQ_OPR)
    {
        Oid eq_opr = InvalidOid;
        /* BTREE */
        if (typentry->btree_opf != InvalidOid)
            eq_opr = get_opfamily_member(typentry->btree_opf,
                                         typentry->btree_opintype,
                                         typentry->btree_opintype,
                                         BTEqualStrategyNumber);
        /* HASH */
        if (typentry->hash_opf != InvalidOid && eq_opr == InvalidOid)
            eq_opr = get_opfamily_member(typentry->hash_opf,
                                         typentry->hash_opintype,
                                         typentry->hash_opintype,
                                         HTEqualStrategyNumber);
    }
}
```

```
static Datum
ExecInterpExpr(ExprState *state, ExprContext *econtext)
{
    EEO_CASE(EEOP_NOT_DISTINCT)
    {
        if (left_isnull && right_isnull)
        {
            *op->resvalue = true;
        }
        else if (left_isnull || right_isnull)
        {
            *op->resvalue = false;
        }
        else
        {
            *op->resvalue = eqfunction();
        }
    }
}
```

HashAggregate

```
EXPLAIN SELECT a, b FROM tbl GROUP BY a, b;

     QUERY PLAN
------------------------
HashAggregate
  Group Key: a, b
  ->  Seq Scan on tbl
```


bucket
..0110 0b11
hash(key) & mask = **0b10**
0b00
0b01
0b10
0b11
...01**00**
AggStatePerGroup
...00**00**
AggStatePerGroup
...01**10**
AggStatePerGroup

```
static int
hash_choose_num_partitions(double input_groups, double hashentrysize,
                           int used_bits, int *log2_npartitions)
{
    Size        hash_mem_limit = get_hash_memory_limit();

    double mem_wanted = input_groups * hashentrysize;

    /* make enough partitions so that each one is likely to fit in memory */
    double dpartitions = 1 + (mem_wanted / hash_mem_limit);
}
```

```
static void
hashagg_spill_init(HashAggSpill *spill, LogicalTapeSet *tapeset, int used_bits,
                   double input_groups, double hashentrysize)
{
   npartitions = hash_choose_num_partitions(input_groups, hashentrysize,
                                            used_bits, &partition_bits);

   spill->shift = 32 - used_bits - partition_bits;
   if (spill->shift < 32)
       spill->mask = (npartitions - 1) << spill->shift;
   else
       spill->mask = 0;
   /* ... */
}
```

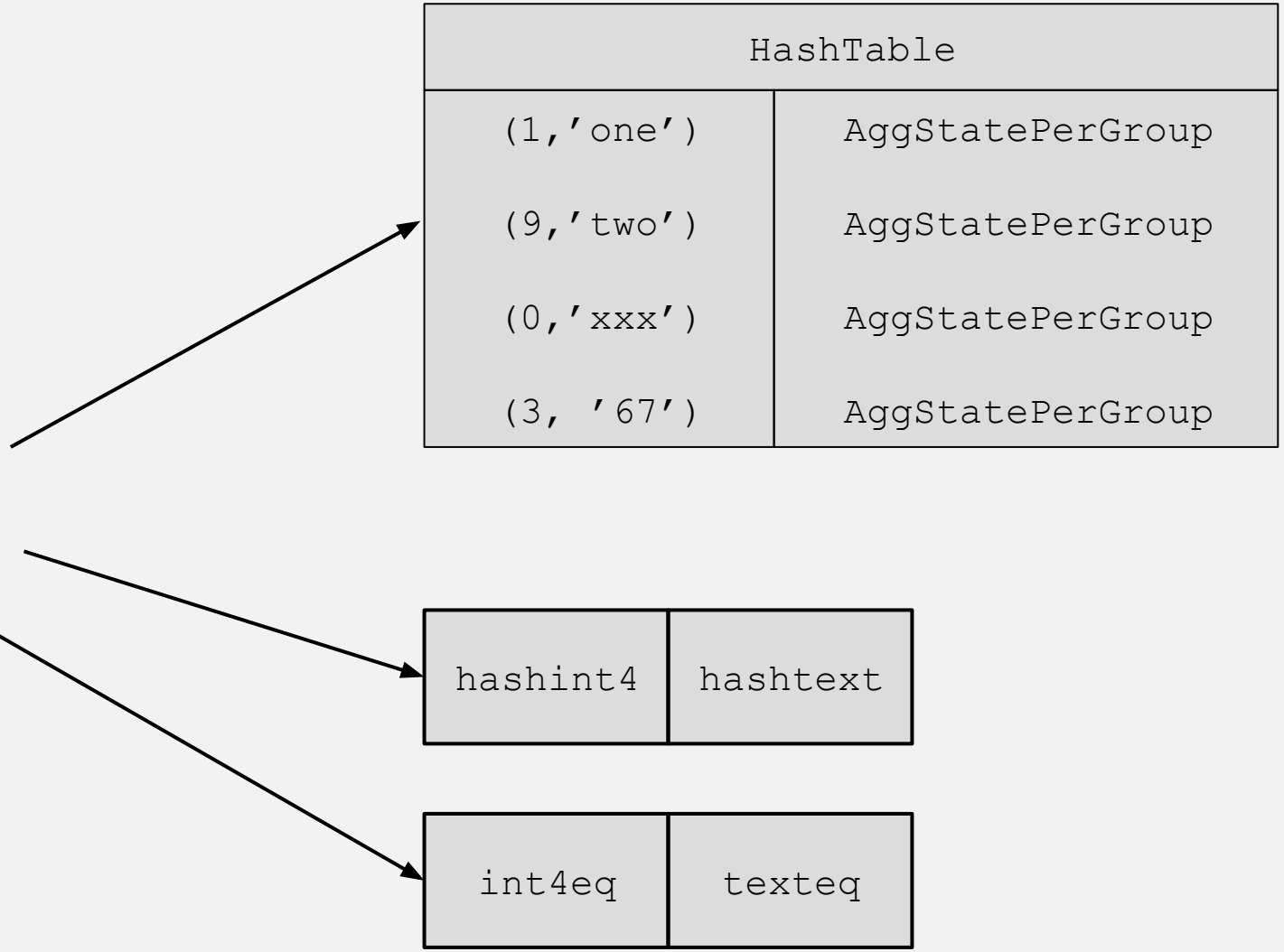
```
static void
hashagg_spill_init(HashAggSpill *spill, LogicalTapeSet *tapeset, int used_bits,
                   double input_groups, double hashentrysize)
{
   npartitions = hash_choose_num_partitions(input_groups, hashentrysize,
                                            used_bits, &partition_bits);





   /* ... */
   for (int i = 0; i < npartitions; i++)
       initHyperLogLog(&spill->hll_card[i], HASHAGG_HLL_BIT_WIDTH);
}
```

```
static TupleTableSlot *ExecAgg(PlanState *pstate)
{
    AggState *node = castNode(AggState, pstate);
    switch (node->aggstrategy)
    {
        case AGG_HASHED:
            if (!node->table_filled)
                agg_fill_hash_table(node);
            result = agg_retrieve_hash_table(node);
        case AGG_MIXED:
        case AGG_SORTED:
        case AGG_PLAIN:
            /* ... */
    }
}
```

```
static void
agg_fill_hash_table(AggState *aggstate)
{
    TupleTableSlot *outerslot;
    for (;;)
    {
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
            break;
        lookup_hash_entries(aggstate);
        advance_aggregates(aggstate);
    }

    aggstate->table_filled = true;
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    AggStatePerHash perhash = aggstate->perhash;
    TupleHashTable hashtable = perhash->hashtable;
    bool isnew = false;
    TupleHashEntry entry;

    entry = LookupTupleHashEntry(hashtable, hashslot, &isnew, &hash);
    if (isnew)
        initialize_hash_entry(aggstate, hashtable, entry);

    aggstate->pergroup = TupleHashEntryGetAdditional(hashtable, entry);
}
```

```
struct AggStatePerHash
{
    TupleHashTable hashtable;
    FmgrInfo  *hashfunctions;
    Oid       *eqfuncoids;
    int        numCols;
}
```



```
static void
lookup_hash_entries(AggState *aggstate)
{
    AggStatePerHash perhash = aggstate->perhash;
    TupleHashTable hashtable = perhash->hashtable;
    bool isnew = false;
    TupleHashEntry entry;

    entry = LookupTupleHashEntry(hashtable, hashslot, &isnew, &hash);
    if (isnew)
        initialize_hash_entry(aggstate, hashtable, entry);

    aggstate->pergroup = TupleHashEntryGetAdditional(hashtable, entry);
}
```

```
TupleHashEntry
LookupTupleHashEntry(TupleHashTable hashtable, TupleTableSlot *slot,
                     bool *isnew, uint32 *hash)
{
    *hash = TupleHashTableHash_internal(hashtable->hashtab, NULL);
    entry = LookupTupleHashEntry_internal(hashtable, slot, isnew, *hash);
    return entry;
}
```

```
static uint32
TupleHashTableHash_internal(struct tuplehash_hash *tb, const MinimalTuple tuple)
{
    uint32 hashkey;
    hashkey = ExecEvalExpr(hashtable->tab_hash_expr, hashtable->exprcontext,
                           &isnull));
    return murmurhash32(hashkey);
}
```

```
#define HASHSTANDARD_PROC   1

bool get_op_hash_functions(Oid opno)
{
   CatCList *catlist = SearchSysCacheList1(AMOPOPID, ObjectIdGetDatum(opno));
   for (i = 0; i < catlist->n_members; i++)
   {
       Form_pg_amop aform = (Form_pg_amop) GETSTRUCT(tuple);
       if (aform->amopmethod == HASH_AM_OID &&
           aform->amopstrategy == HTEqualStrategyNumber)
       {
           return get_opfamily_proc(aform->amopfamily,
                                    aform->amoplefttype,
                                    aform->amoplefttype,
                                   HASHSTANDARD_PROC);
       }
   }
}
```

```
static uint32
TupleHashTableHash_internal(struct tuplehash_hash *tb, const MinimalTuple tuple)
{
    uint32 hashkey;
    hashkey = ExecEvalExpr(hashtable->tab_hash_expr, hashtable->exprcontext,
                           &isnull));
    return murmurhash32(hashkey);
}
```

```
static TupleHashEntry
LookupTupleHashEntry_internal( TupleHashTable hashtable, TupleTableSlot *slot,
                               bool *isnew, uint32 hash)
{
   TupleHashEntryData *entry;
   bool found;

   entry = tuplehash_insert_hash(hashtable->hashtab, hash, &found);
   if (found)
       *isnew = false;
   else
   {
       *isnew = true;
        entry->firstTuple = ExecCopySlotMinimalTupleExtra(slot,
                                                          hashtable->additionalsize);
   }
   return entry;
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    AggStatePerHash perhash = aggstate->perhash;
    TupleHashTable hashtable = perhash->hashtable;
    bool isnew = false;
    TupleHashEntry entry;

    entry = LookupTupleHashEntry(hashtable, hashslot, &isnew, &hash);
    if (isnew)
        initialize_hash_entry(aggstate, hashtable, entry);

    aggstate->pergroup = TupleHashEntryGetAdditional(hashtable, entry);
}
```

```
static void
initialize_hash_entry(AggState *aggstate, TupleHashTable hashtable,
                      TupleHashEntry entry)
{
    for (transno = 0; transno < aggstate->numtrans; transno++)
    {
        AggStatePerTrans pertrans = &aggstate->pertrans[transno];
        AggStatePerGroup pergroupstate = &pergroup[transno];

        initialize_aggregate(aggstate, pertrans, pergroupstate);
    }
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    AggStatePerHash perhash = aggstate->perhash;
    TupleHashTable hashtable = perhash->hashtable;
    bool isnew = false;
    TupleHashEntry entry;

    entry = LookupTupleHashEntry(hashtable, hashslot, &isnew, &hash);
    if (isnew)
        initialize_hash_entry(aggstate, hashtable, entry);

    aggstate->pergroup = TupleHashEntryGetAdditional(hashtable, entry);
}
```

```
static void
agg_fill_hash_table(AggState *aggstate)
{
    TupleTableSlot *outerslot;
    for (;;)
    {
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
            break;
        lookup_hash_entries(aggstate);
        advance_aggregates(aggstate);
    }

    aggstate->table_filled = true;
}
```

```
static TupleTableSlot *ExecAgg(PlanState *pstate)
{
    AggState *node = castNode(AggState, pstate);
    switch (node->aggstrategy)
    {
        case AGG_HASHED:
            if (!node->table_filled)
                agg_fill_hash_table(node);
            result = agg_retrieve_hash_table(node);
        case AGG_MIXED:
        case AGG_SORTED:
        case AGG_PLAIN:
            /* ... */
    }
}
```

```
static TupleTableSlot *
agg_retrieve_hash_table_in_memory(AggState *aggstate)
{
    for (;;)
    {
        TupleHashTable hashtable = perhash->hashtable;
        TupleHashEntry entry;
        entry = ScanTupleHashTable(hashtable, &perhash->hashiter);
        if (entry == NULL)
            return NULL;
        finalize_aggregates(aggstate, peragg, pergroup);
        result = project_aggregates(aggstate);
        if (result)
            return result;
    }
}
```

```
static void
initialize_hash_entry(AggState *aggstate, TupleHashTable hashtable,
                      TupleHashEntry entry)
{
    hash_agg_check_limits(aggstate);

    /* ... инициализация агрегатов ... */
}
```

```
static void
hash_agg_check_limits(AggState *aggstate)
{
    /* память хэш-таблицы */
    Size meta_mem = MemoryContextMemAllocated(hash_metacxt);
    Size entry_mem = MemoryContextMemAllocated(hash_tablecxt);
    /* состояние агрегатов */
    Size tval_mem = MemoryContextMemAllocated(ecxt_per_tuple_memory);

    Size total_mem = meta_mem + entry_mem + tval_mem;
    if (total_mem > aggstate->hash_mem_limit)
        hash_agg_enter_spill_mode(aggstate);
}
```

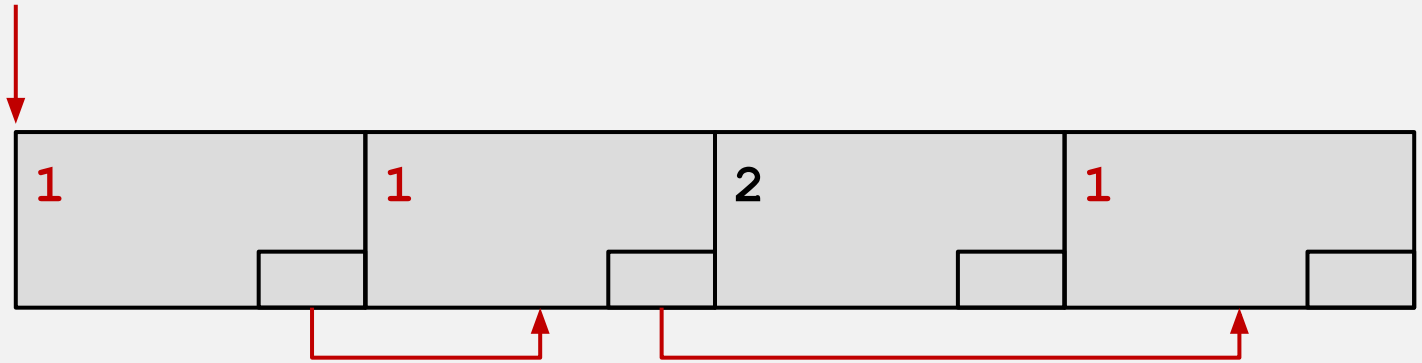
```
static void
hash_agg_enter_spill_mode(AggState *aggstate)
{
   aggstate->hash_spill_mode = true;
   hashagg_recompile_expressions(aggstate, aggstate->table_filled, true);
   aggstate->hash_tapeset = LogicalTapeSetCreate(true, NULL, -1);
   HashAggSpill *spill = palloc(sizeof(HashAggSpill));
   hashagg_spill_init(spill, aggstate->hash_tapeset, 0,
                      perhash->aggnode->numGroups,
                      aggstate->hashentrysize);
}
```

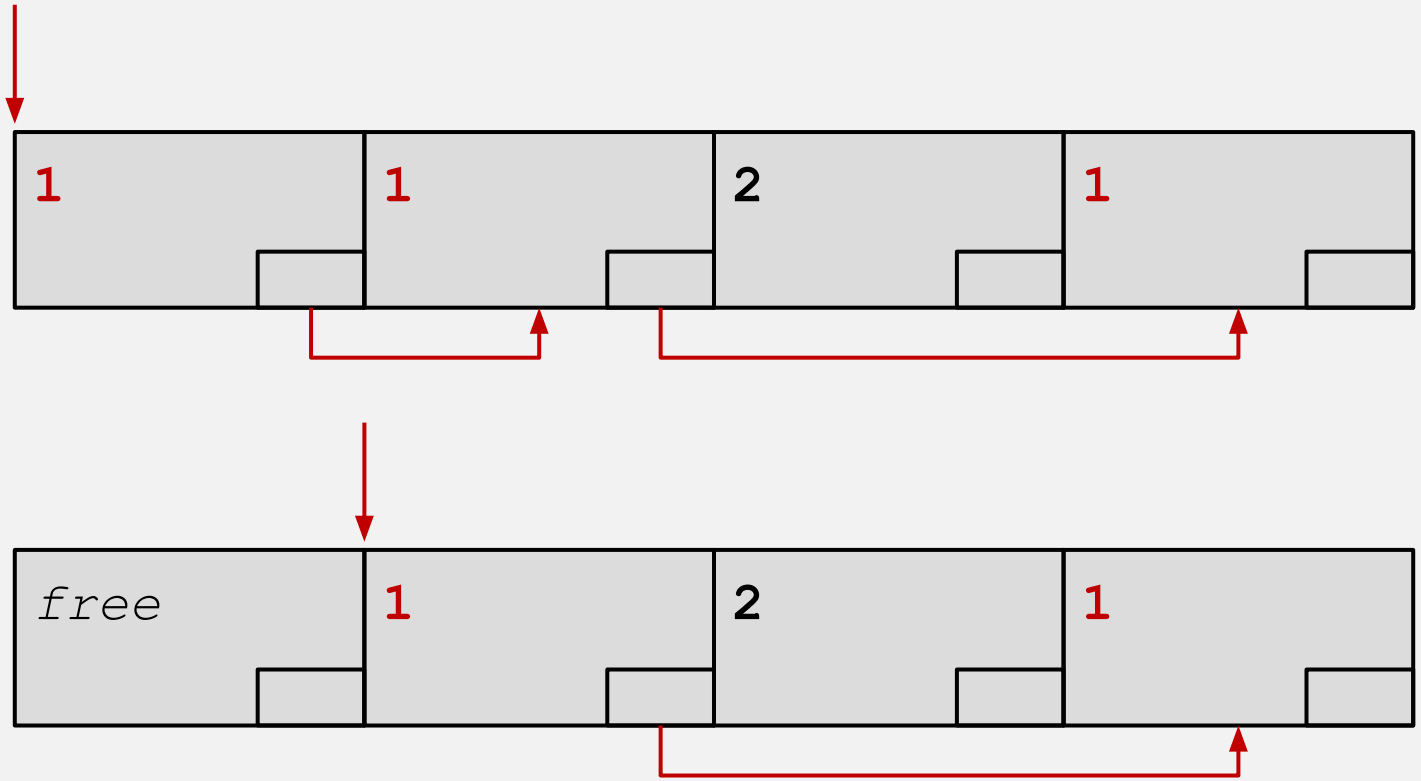
```
static void
lookup_hash_entries(AggState *aggstate)
{
    TupleHashEntry entry;

    entry = LookupTupleHashEntry();
    if (entry != NULL)
    {
        aggstate->pergroup = TupleHashEntryGetAdditional(entry);
    }
    else
    {
        aggstate->pergroup = NULL;
    }
}
```

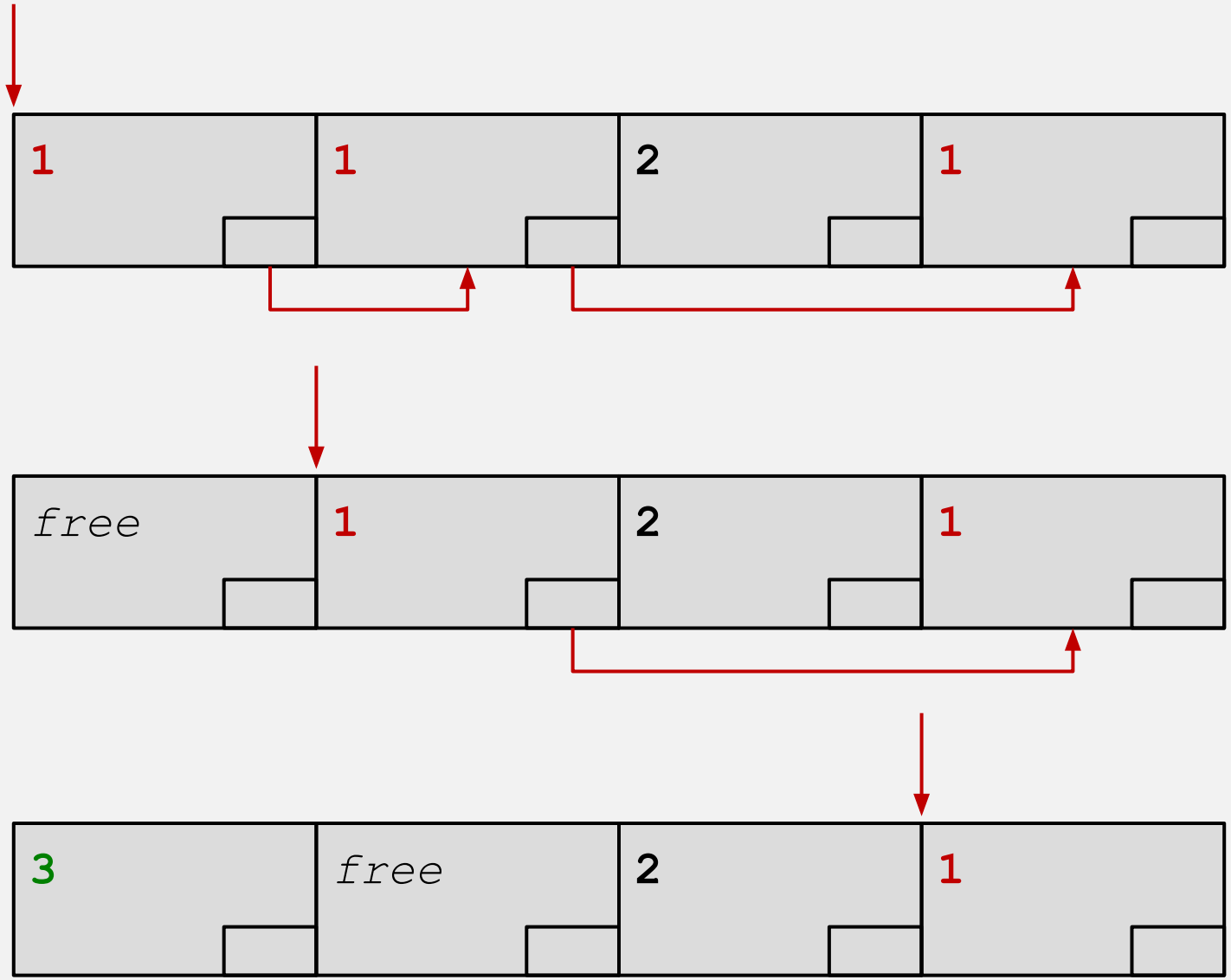
```
static void
hash_agg_enter_spill_mode(AggState *aggstate)
{
   aggstate->hash_spill_mode = true;
   hashagg_recompile_expressions(aggstate, aggstate->table_filled, true);
   aggstate->hash_tapeset = LogicalTapeSetCreate(true, NULL, -1);
   HashAggSpill *spill = palloc(sizeof(HashAggSpill));
   hashagg_spill_init(spill, aggstate->hash_tapeset, 0,
                      perhash->aggnode->numGroups,
                      aggstate->hashentrysize);
}
```

```c
static void
ExecBuildAggTransCall(ExprState *state, AggState *aggstate, bool nullcheck)
{
    /* ... */
    /* add check for NULL pointer? */
    if (nullcheck)
    {
        scratch->opcode = EEOP_AGG_PLAIN_PERGROUP_NULLCHECK;
        scratch->d.agg_plain_pergroup_nullcheck.setoff = setoff;
        /* adjust later */
        scratch->d.agg_plain_pergroup_nullcheck.jumpnull = - 1;
        ExprEvalPushStep(state, scratch);
        adjust_jumpnull = state->steps_len -  1;
    }
    /* ... */
}
```

```
static void
hash_agg_enter_spill_mode(AggState *aggstate)
{
   aggstate->hash_spill_mode = true;
   hashagg_recompile_expressions(aggstate, aggstate->table_filled, true);
   aggstate->hash_tapeset = LogicalTapeSetCreate(true, NULL, -1);
   HashAggSpill *spill = palloc(sizeof(HashAggSpill));
   hashagg_spill_init(spill, aggstate->hash_tapeset, 0,
                      perhash->aggnode->numGroups,
                      aggstate->hashentrysize);
}
```


1
1
2
1


1
1
2
1
free
1
2
1


1
1
2
1
free
1
2
1
3
free
2
1

```
static void
hash_agg_enter_spill_mode(AggState *aggstate)
{
   aggstate->hash_spill_mode = true;
   hashagg_recompile_expressions(aggstate, aggstate->table_filled, true);
   aggstate->hash_tapeset = LogicalTapeSetCreate(true, NULL, -1);
   HashAggSpill *spill = palloc(sizeof(HashAggSpill));
   hashagg_spill_init(spill, aggstate->hash_tapeset,
                      0,
                     perhash->aggnode->numGroups,
                     aggstate->hashentrysize);
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    bool  isnew = false;
    bool *p_isnew = aggstate->hash_spill_mode ? NULL : &isnew;

    entry = LookupTupleHashEntry(hashtable, hashslot,
                                 p_isnew, &hash);
    if (entry == NULL)
    {
        HashAggSpill *spill = &aggstate->hash_spills;
        hashagg_spill_tuple(aggstate, spill, slot, hash);
        aggstate->pergroup = NULL;
    }
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    bool  isnew = false;
    bool *p_isnew = aggstate->hash_spill_mode ? NULL : &isnew;

    entry = LookupTupleHashEntry(hashtable, hashslot,
                                 p_isnew, &hash);
    if (entry == NULL)
    {
        HashAggSpill *spill = &aggstate->hash_spills;
        hashagg_spill_tuple(aggstate, spill, slot, hash);
        aggstate->pergroup = NULL;
    }
}
```

```
EXPLAIN VERBOSE SELECT a FROM tbl GROUP BY a;

                QUERY PLAN
-------------------------------------
HashAggregate
  Output: a
  Group Key: tbl.a
  ->  Seq Scan on public.tbl
        Output: a, b, c
(5 rows)
```

```
static Size
hashagg_spill_tuple(AggState *aggstate, TupleTableSlot *inputslot)
{
    if (!aggstate->all_cols_needed)
    {
        TupleTableSlot *spillslot = aggstate->hash_spill_wslot;
        for (int i = 0; i < natts; i++)
            if (bms_is_member(i + 1, aggstate->colnos_needed))
            {
                spillslot->tts_values[i] = inputslot->tts_values[i];
                spillslot->tts_isnull[i] = inputslot->tts_isnull[i];
            }
            else
                spillslot->tts_isnull[i] = true;
    }
    /* ... */
}
```

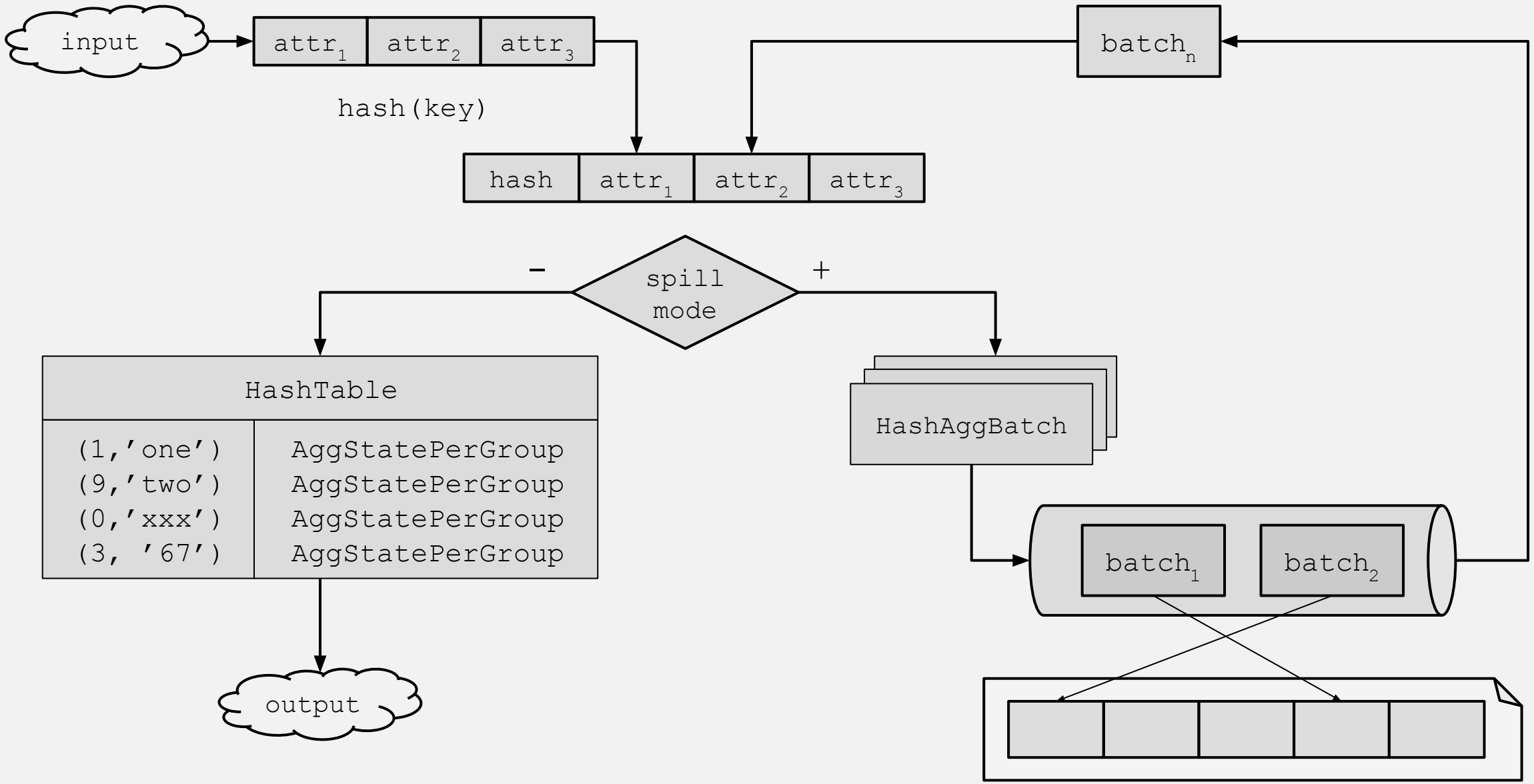
```
static Size
hashagg_spill_tuple(AggState *aggstate, TupleTableSlot *inputslot)
{
    /* ... */

    if (spill->shift < 32)
        partition = (hash & spill->mask) >> spill->shift;
    else
        partition = 0;

    spill->ntuples[partition]++;
    addHyperLogLog(&spill->hll_card[partition], hash_bytes_uint32(hash));

    /* ... */
}
```

```
static Size
hashagg_spill_tuple(AggState *aggstate, TupleTableSlot *inputslot)
{
    /* ... */

    tape = spill->partitions[partition];

    LogicalTapeWrite(tape, &hash, sizeof(uint32));
    total_written += sizeof(uint32);

    LogicalTapeWrite(tape, tuple, tuple->t_len);
    total_written += tuple->t_len;
    return total_written;
}
```

```
static void
lookup_hash_entries(AggState *aggstate)
{
    bool  isnew = false;
    bool *p_isnew = aggstate->hash_spill_mode ?  NULL : &isnew;

    entry = LookupTupleHashEntry(hashtable, hashslot, p_isnew, &hash);
    if (entry == NULL)
    {
        HashAggSpill *spill = &aggstate->hash_spills;
        hashagg_spill_tuple(aggstate, spill, slot, hash);
        aggstate->pergroup = NULL;
    }
}
```

```
static void
agg_fill_hash_table(AggState *aggstate)
{
    TupleTableSlot *outerslot;
    for (;;)
    {
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
            break;
        lookup_hash_entries(aggstate);
        advance_aggregates(aggstate);
    }

    hashagg_finish_initial_spills(aggstate);
    aggstate->table_filled = true;
}
```

```
static void
hashagg_spill_finish(AggState *aggstate, HashAggSpill *spill)
{
    for (i = 0; i < spill->npartitions; i++)
    {
        LogicalTape *tape = spill->partitions[i];
        HashAggBatch *new_batch;
        double      cardinality;

        cardinality = estimateHyperLogLog(&spill->hll_card[i]);
        freeHyperLogLog(&spill->hll_card[i]);
        LogicalTapeRewindForRead(tape);

        new_batch = hashagg_batch_new(tape, spill->ntuples[i], cardinality, used_bits);
        aggstate->hash_batches = lappend(aggstate->hash_batches, new_batch);
    }
}
```


input
attr1
attr2
attr3
hash(key)
batchn
hash
attr1
attr2
attr3
spill mode
–
+
HashTable
(1,'one')
(9,'two')
(0,'xxx')
(3, '67')
AggStatePerGroup
AggStatePerGroup
AggStatePerGroup
AggStatePerGroup
HashAggBatch
batch1
batch2
output

```
static bool
agg_refill_hash_table(AggState *aggstate)
{
   for (;;)
   {
      bool  isnew = false;
      bool *p_isnew = aggstate->hash_spill_mode ? NULL : &isnew;
      tuple = hashagg_batch_read(batch, &hash);
      if (tuple == NULL)
          break;
      entry = LookupTupleHashEntryHash(hashtable, hashslot, p_isnew, hash);
      if (entry != NULL)
         /* initialize_hash_entry, advance_aggregates */
      else
         /* hashagg_spill_tuple */
   }
}
```

```
static bool
agg_refill_hash_table(AggState *aggstate)
{
   for (;;)
   {
      bool  isnew = false;
      bool *p_isnew = aggstate->hash_spill_mode ? NULL : &isnew;
      tuple = hashagg_batch_read(batch, &hash);
      if (tuple == NULL)
          break;
      entry = LookupTupleHashEntryHash(hashtable, hashslot, p_isnew, hash);
      if (entry != NULL)
         /* initialize_hash_entry, advance_aggregates */
      else
         /* hashagg_spill_tuple */
   }
}
```

```
static bool
agg_refill_hash_table(AggState *aggstate)
{
    for (;;)
    {
        entry = LookupTupleHashEntryHash(hashtable, hashslot, p_isnew, hash);
        if (entry == NULL)
        {
            if (!spill_initialized)
            {
                spill_initialized = true;
                hashagg_spill_init(&spill, tapeset,
                                   batch->used_bits,
                                   batch->input_card,
                                   aggstate->hashentrysize);
            }
        }
    }
}
```

GROUPING SETS
CUBE
ROLLUP

```
EXPLAIN SELECT a, b, c FROM tbl
    GROUP BY GROUPING SETS(a, (b, d), ROLLUP(a, b, c));

          QUERY PLAN
--------------------------------
MixedAggregate
  Hash Key: a, b, c
  Hash Key: a, b
  Hash Key: a
  Hash Key: a
  Hash Key: b, d
  Group Key: ()
  ->  Seq Scan on tbl
```

```
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY GROUPING SETS ((brand), (size), ());
```

```
        QUERY PLAN
-----------------------------
MixedAggregate
  Hash Key: brand
  Hash Key: size
  Group Key: ()
  ->  Seq Scan on items_sold
```

brand	size	sum
Foo		30
Bar		20
	L	15
	M	35
		50

```
SELECT brand, NULL, sum(sales)
     FROM items_sold
     GROUP BY brand

UNION ALL

SELECT NULL, size, sum(sales)
     FROM items_sold
     GROUP BY size

UNION ALL

SELECT NULL, NULL, sum(sales)
     FROM items_sold
     GROUP BY ();
```

```sql
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY ROLLUP (brand, size);
```

↓

```sql
ROLLUP(a, b, c)

GROUPING SETS(
   (a, b, c),
   (a, b),
   (a),
   ()
)
```

```sql
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY GROUPING SETS (
           (brand, size),
           (brand),
           ()
  );
```

```
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY ROLLUP (brand, size);
```

```
           QUERY PLAN
-----------------------------------
MixedAggregate
  Hash Key: brand, size
  Hash Key: brand
  Group Key: ()
  ->  Seq Scan on items_sold
```

brand	size	sum
		50
Foo	M	20
Bar	L	5
Bar	M	15
Foo	L	10
Foo		30
Bar		20

```
CUBE(a, b, c)

GROUPING SETS(
    (a, b, c),
    (a, b   ),
    (a,    c),
    (a      ),
    (   b, c),
    (   b   ),
    (      c),
    (       )
)
```

```
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY CUBE (brand, size);
```

↓

```
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY GROUPING SETS (
      (brand, size),
      (brand      ),
      (       size),
      (           )
  );
```

```
SELECT brand, size, sum(sales)
  FROM items_sold
  GROUP BY CUBE (brand, size);
```

```
        QUERY PLAN
-------------------------------
MixedAggregate
  Hash Key: brand, size
  Hash Key: brand
  Hash Key: size
  Group Key: ()
  ->  Seq Scan on items_sold
(6 rows)
```

brand	size	sum
		50
Foo	M	20
Bar	L	5
Bar	M	15
Foo	L	10
Foo		30
Bar		20
	L	15
	M	35

```
SELECT a, b FROM tbl GROUP BY a, b;

SELECT a, b FROM tbl GROUP BY GROUPING SETS((a, b));
```

```
        QUERY PLAN
------------------------
HashAggregate
  Group Key: a, b
  ->  Seq Scan on tbl
```

```
SELECT a, b FROM tbl
   GROUP BY GROUPING SETS(a, b), ROLLUP(a, c);

        QUERY PLAN
--------------------------
HashAggregate
  Hash Key: a, c, b
  Hash Key: a, c
  Hash Key: a
  Hash Key: a
  Hash Key: b, a
  Hash Key: b
  ->  Seq Scan on tbl
```

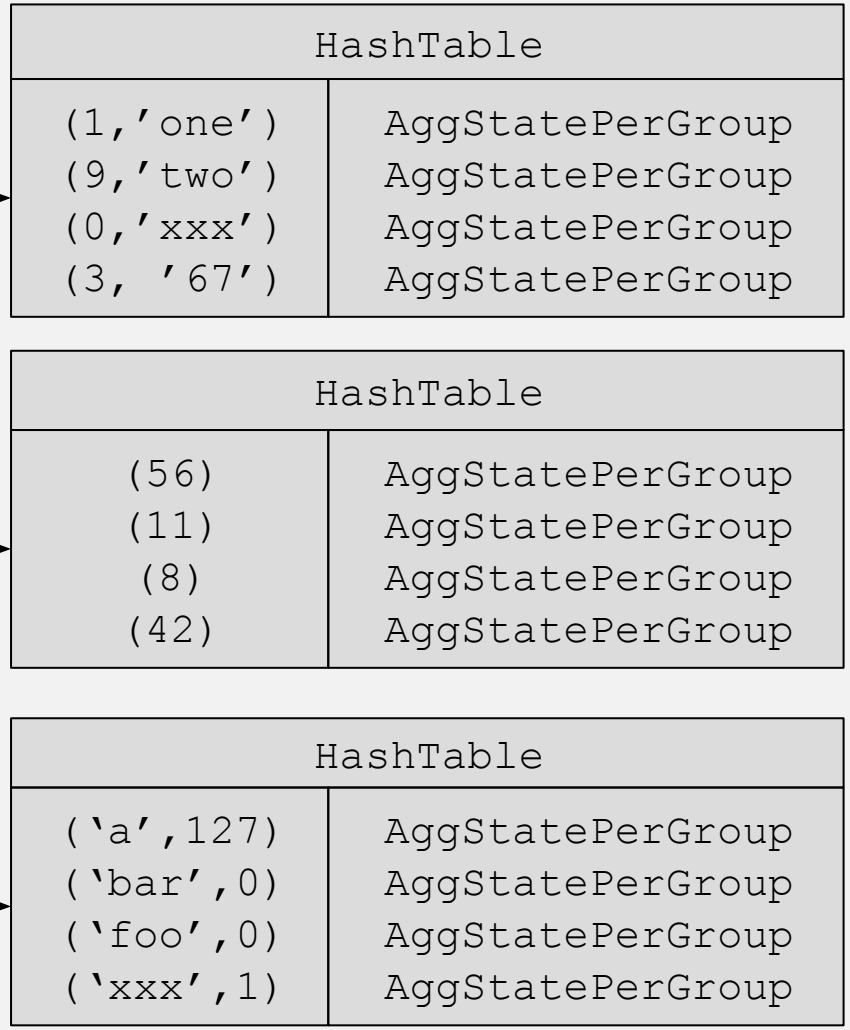
GS		ROLLUP		
				(a, c)
(a)		(a, c)		(a)
	x	(a)	=	(a)
(b)		()		(b, a, c)
				(b, a)
				(b)

```
SELECT a, b FROM tbl GROUP BY
    GROUPING SETS(a, ROLLUP(a, b, (a, b)));
```

```
SELECT a, b FROM tbl GROUP BY
    ROLLUP(a, b, ROLLUP(a, b, c));
```

```
SELECT a, b FROM tbl
  GROUP BY GROUPING SETS((a, b), c, (c, d));
```

```
       QUERY PLAN
-----------------------
HashAggregate
  Hash Key: c, d
  Hash Key: c
  Hash Key: a, b
  ->  Seq Scan on tbl
(5 rows)
```



```
static void
lookup_hash_entries(AggState *aggstate)
{
    for (int setno = 0; setno < aggstate->num_hashes; setno++)
    {
        AggStatePerHash perhash = &aggstate->perhash[setno];
        entry = LookupTupleHashEntry(hashtable, hashslot, p_isnew, &hash);
        if (entry != NULL)
        {
            /* ... */
            pergroup[setno] = TupleHashEntryGetAdditional(hashtable, entry);
        }
        else
        {
            /* ... */
            pergroup[setno] = NULL;
        }
    }
}
```

```
static void
hash_agg_enter_spill_mode(AggState *aggstate)
{
    for (int setno = 0; setno < aggstate->num_hashes; setno++)
    {
        AggStatePerHash perhash = &aggstate->perhash[setno];
        HashAggSpill *spill = &aggstate->hash_spills[setno];

        hashagg_spill_init(spill, aggstate->hash_tapeset, 0,
                           perhash->aggnode->numGroups,
                           aggstate->hashentrysize);
    }
}
```

```
static void
hashagg_finish_initial_spills(AggState *aggstate)
{
    for (int setno = 0; setno < aggstate->num_hashes; setno++)
    {
        HashAggSpill *spill = &aggstate->hash_spills[setno];
        hashagg_spill_finish(aggstate, spill, setno);
    }
}
```

```
SELECT a, b, c FROM tbl GROUP BY ROLLUP(a, b, c);
```

```
GROUPING SETS(
    (a, b, c),
    (a, b   ),
    (a      ),
    (       )
)
```

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

Финализация

1	1	2

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

Финализация

1	1	2

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	1	2

Финализация

1	1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

Финализация

1	1	2

1	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

1	2	1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Представитель

2	2	1

Финализация

1	2	1

1	2

1

1	1	1
1	1	1
1	1	2
1	2	1
1	2	1
2	2	1

Финализация

2	2	1
2	2	
2		

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
    for (;;)
    {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (node->aggstrategy != AGG_PLAIN)
        {
            if (!ExecQual(aggstate->eqfunctions[node->numCols-1], tmpcontext))
            {
                aggstate->grp_firstTuple = ExecCopySlotHeapTuple(outerslot);
                break;
            }
        }
    }
    finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   if (node->aggstrategy != AGG_PLAIN &&
       aggstate->projected_set != -1 &&
       aggstate->projected_set < (numGroupingSets -1) &&
       !ExecQualAndReset(aggstate->phase->eqfunctions[nextSetSize-1], tmpcontext))
       aggstate->projected_set += 1;
   else
   {
       aggstate->projected_set = 0;
       /* Обработка новой группы */
   }

   select_current_set(aggstate, aggstate->projected_set,false);
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

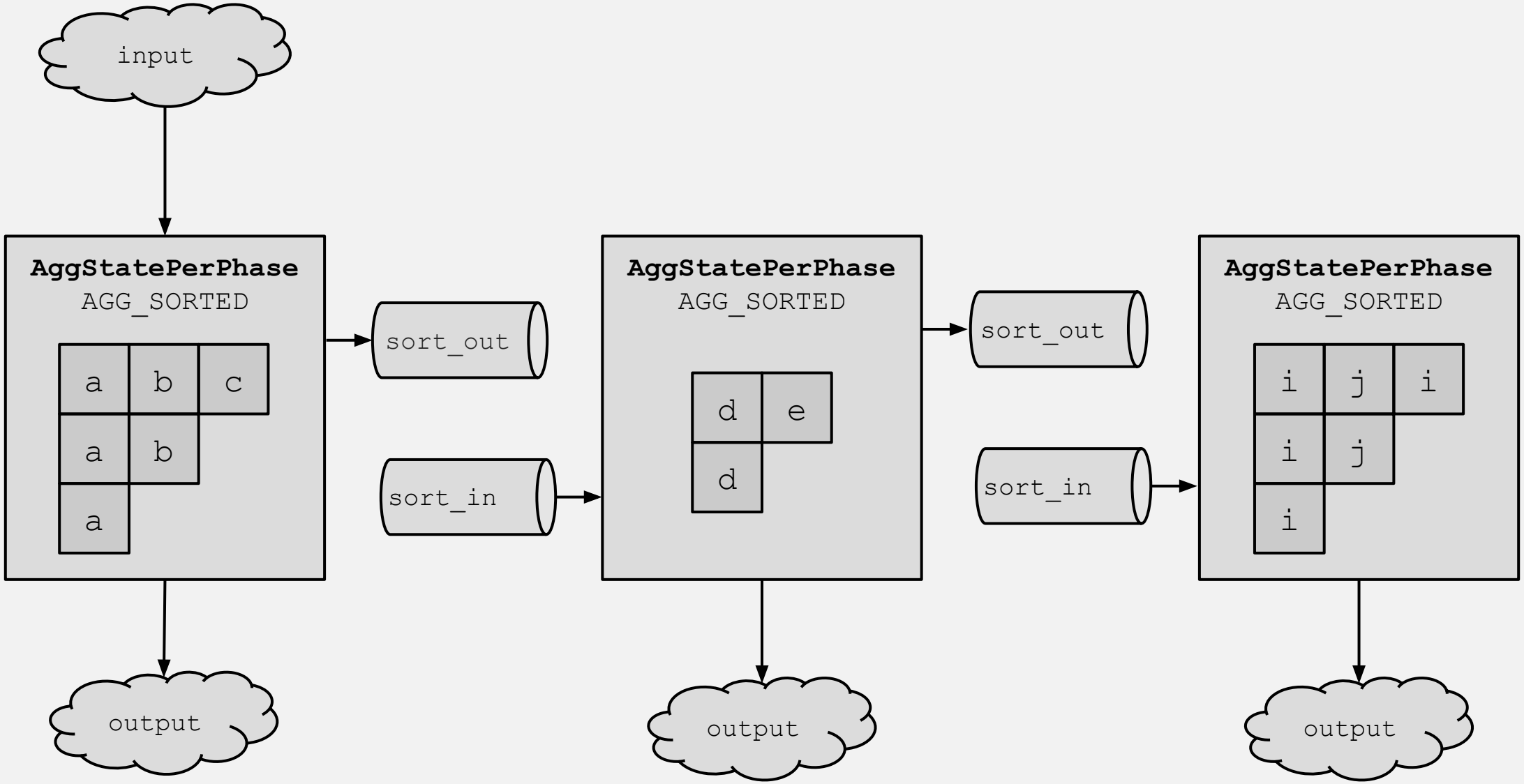
```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   if (node->aggstrategy != AGG_PLAIN &&
       aggstate->projected_set != -1 &&
       aggstate->projected_set < (numGroupingSets -1) &&
       !ExecQualAndReset(aggstate->phase->eqfunctions[nextSetSize-1], tmpcontext))
       aggstate->projected_set += 1;
   else
   {
       aggstate->projected_set = 0;
       /* Обработка новой группы */
   }

   select_current_set(aggstate, aggstate->projected_set,false);
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
SELECT a, b, c FROM tbl
GROUP BY GROUPING SETS (
    (b, a),
    (a),
    (c)
);
```

```
                QUERY PLAN
---------------------------------
GroupAggregate
  Group Key: a, b
  Group Key: a
  Sort Key: c
    Group Key: c
  ->  Sort
        Sort Key: a, b
        ->  Seq Scan on tbl
(8 rows)
```

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
        if (TupIsNull(outerslot))
        {
            aggstate->agg_done = true;
            break;
        }
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static TupleTableSlot *
fetch_input_tuple(AggState *aggstate)
{
    TupleTableSlot *slot;
    if (aggstate->sort_in)
    {
        if (!tuplesort_gettupleslot(aggstate->sort_in, aggstate->sort_slot))
            return NULL;
        slot = aggstate->sort_slot;
    }
    else
        slot = ExecProcNode(outerPlanState(aggstate));

    if (!TupIsNull(slot) && aggstate->sort_out)
        tuplesort_puttupleslot(aggstate->sort_out, slot);
    return slot;
}
```

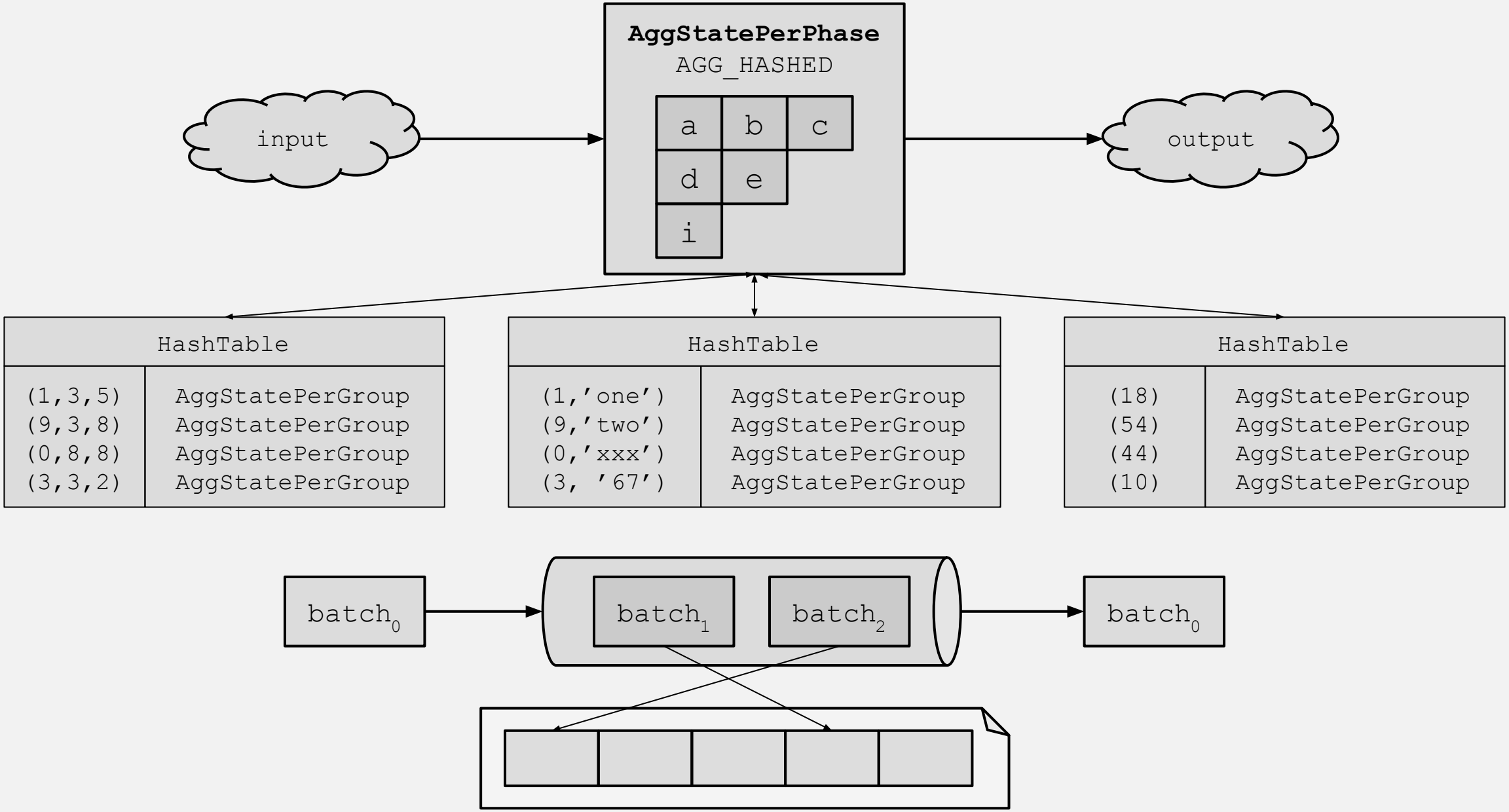

input
AggStatePerPhase
AGG_SORTED
a
b
c
a
b
a
sort_out
sort_in
AggStatePerPhase
AGG_SORTED
d
e
d
sort_out
sort_in
AggStatePerPhase
AGG_SORTED
i
j
i
i
j
i
output
output
output

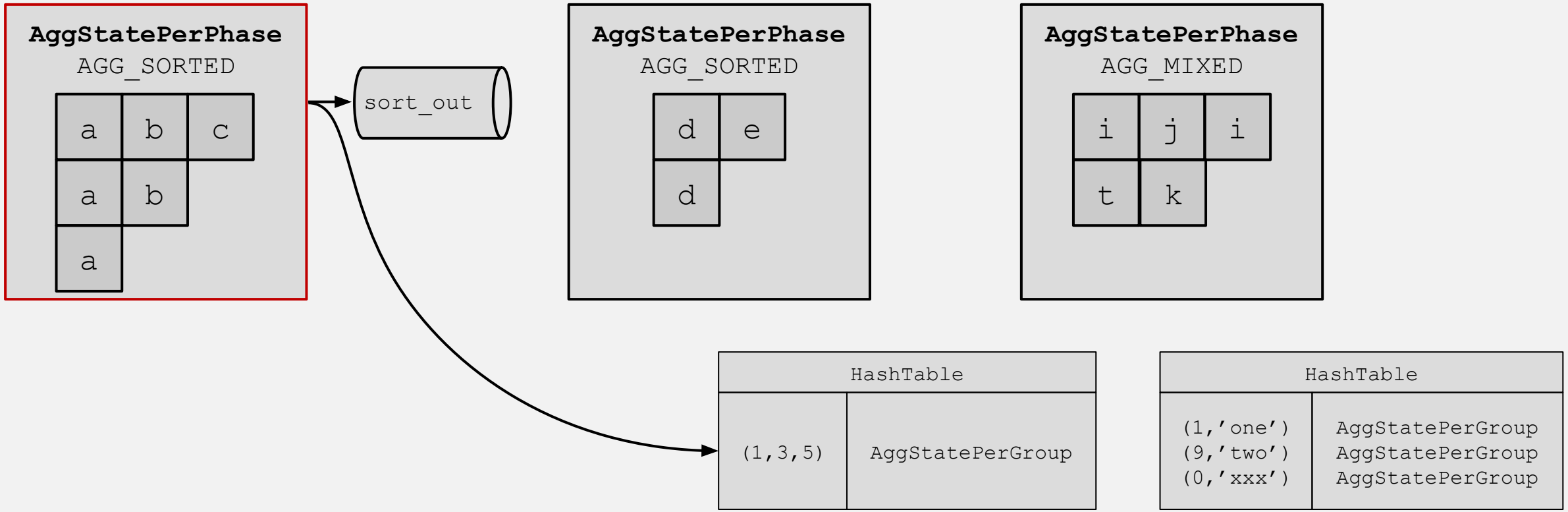
```
static TupleTableSlot *
fetch_input_tuple(AggState *aggstate)
{
    TupleTableSlot *slot;
    if (aggstate->sort_in)
    {
        if (!tuplesort_gettupleslot(aggstate->sort_in, aggstate->sort_slot))
            return NULL;
        slot = aggstate->sort_slot;
    }
    else
        slot = ExecProcNode(outerPlanState(aggstate));

    if (aggstate->sort_out)
        tuplesort_puttupleslot(aggstate->sort_out, slot);
    return slot;
}
```

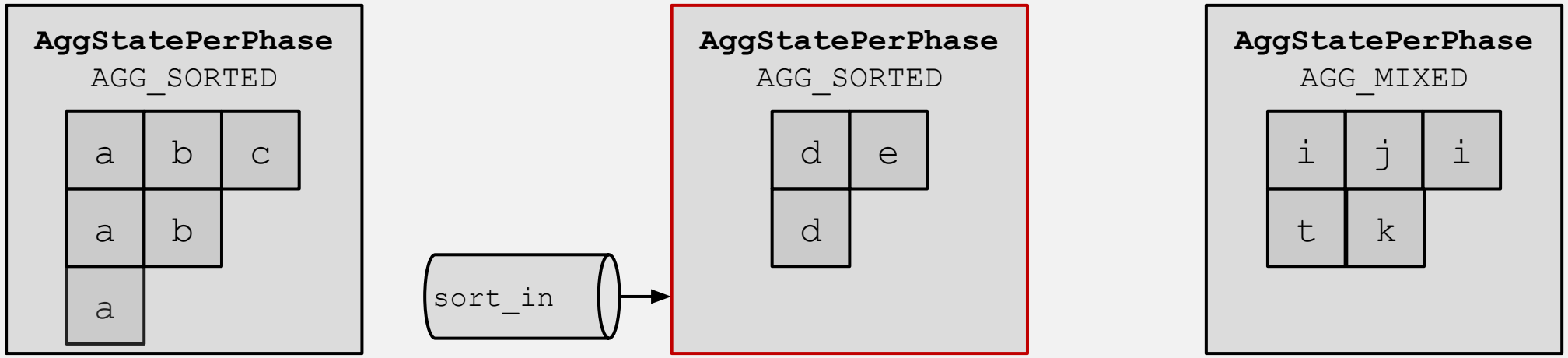
```
static TupleTableSlot *
fetch_input_tuple(AggState *aggstate)
{
    TupleTableSlot *slot;
    if (aggstate->sort_in)
    {
        if (!tuplesort_gettupleslot(aggstate->sort_in, aggstate->sort_slot))
            return NULL;
        slot = aggstate->sort_slot;
    }
    else
        slot = ExecProcNode(outerPlanState(aggstate));

    if (aggstate->sort_out)
        tuplesort_puttupleslot(aggstate->sort_out, slot);
    return slot;
}
```

MixedAggregate

```
SELECT a, b FROM tbl3 GROUP BY CUBE(a, b);

                    QUERY PLAN
-----------------------------------------------
MixedAggregate
  Hash Key: b
  Group Key: a, b
  Group Key: a
  Group Key: ()
  ->  Index Only Scan using tbl3_a_b_idx on tbl3
```

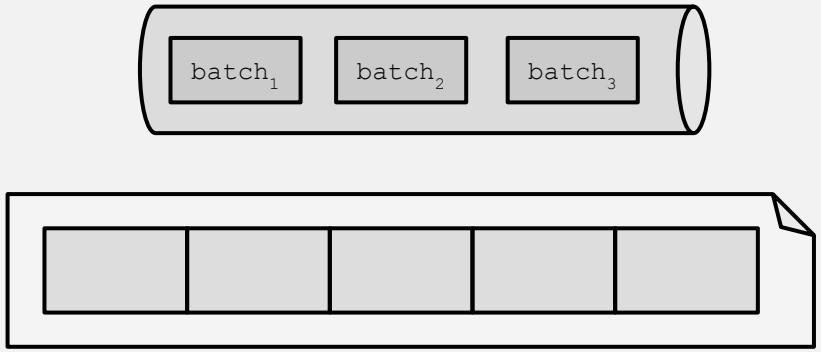

AggStatePerPhase
AGG_HASHED
a
b
c
d
e
i
input
output
HashTable
(1,3,5) AggStatePerGroup
(9,3,8) AggStatePerGroup
(0,8,8) AggStatePerGroup
(3,3,2) AggStatePerGroup
HashTable
(1,'one') AggStatePerGroup
(9,'two') AggStatePerGroup
(0,'xxx') AggStatePerGroup
(3, '67') AggStatePerGroup
HashTable
(18) AggStatePerGroup
(54) AggStatePerGroup
(44) AggStatePerGroup
(10) AggStatePerGroup
batch0
batch1
batch2
batch0

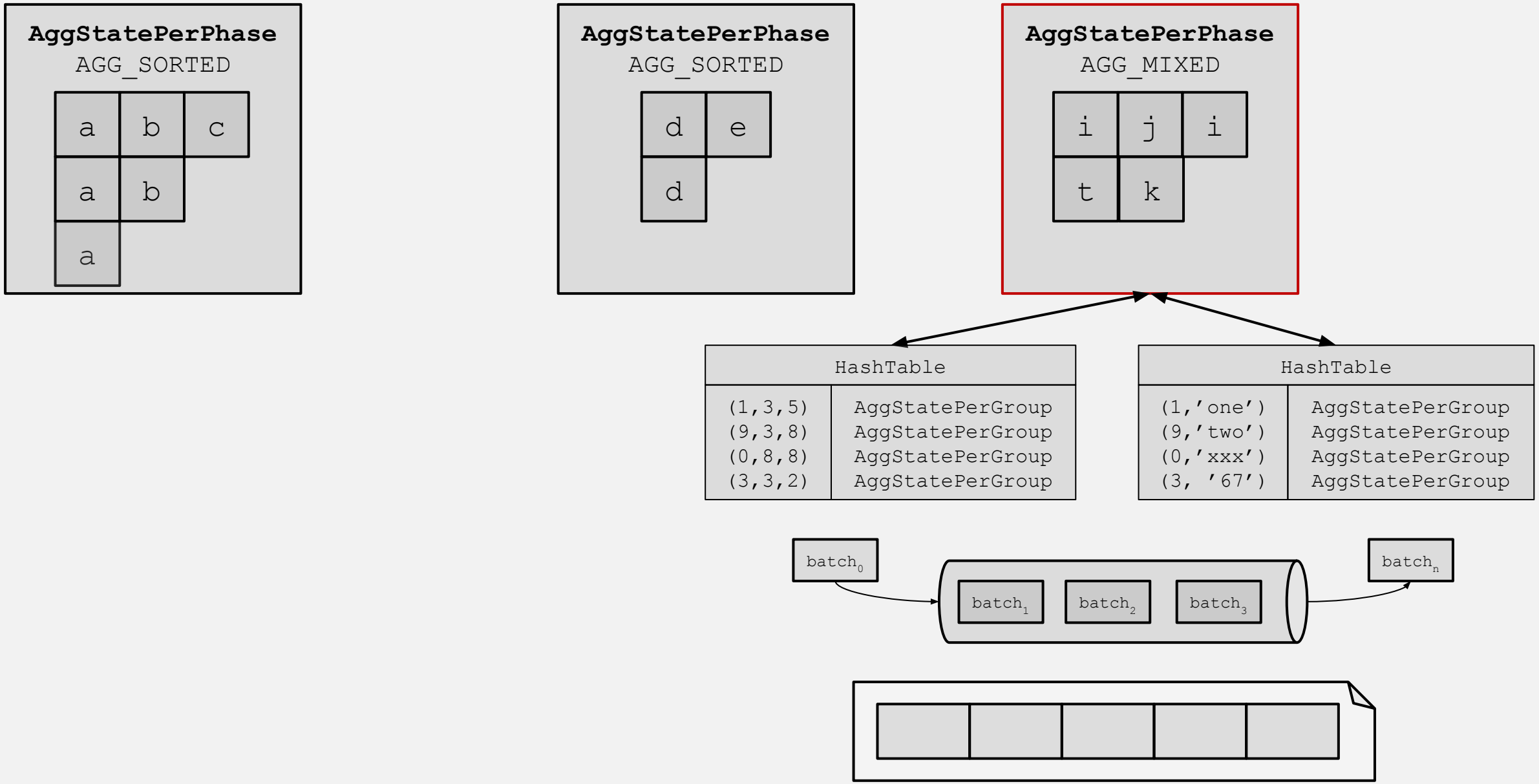

AggStatePerPhase
AGG_SORTED
a b c
a b
a
sort_out
AggStatePerPhase
AGG_SORTED
d e
d
AggStatePerPhase
AGG_MIXED
i j i
t k
HashTable
(1,3,5)
AggStatePerGroup
HashTable
(1,'one')
(9,'two')
(0,'xxx')
AggStatePerGroup
AggStatePerGroup
AggStatePerGroup



HashTable	
(1,3,5)	AggStatePerGroup
(9,3,8)	AggStatePerGroup
(0,8,8)	AggStatePerGroup
(3,3,2)	AggStatePerGroup

HashTable	
(1,'one')	AggStatePerGroup
(9,'two')	AggStatePerGroup
(0,'xxx')	AggStatePerGroup
(3, '67')	AggStatePerGroup




AggStatePerPhase
AGG_SORTED
a b c
a b
a
AggStatePerPhase
AGG_SORTED
d e
d
AggStatePerPhase
AGG_MIXED
i j i
t k
HashTable
(1,3,5) AggStatePerGroup
(9,3,8) AggStatePerGroup
(0,8,8) AggStatePerGroup
(3,3,2) AggStatePerGroup
HashTable
(1,'one') AggStatePerGroup
(9,'two') AggStatePerGroup
(0,'xxx') AggStatePerGroup
(3, '67') AggStatePerGroup
batch0
batch1
batch2
batch3
batchn

```
static TupleTableSlot *agg_retrieve_direct(AggState *aggstate)
{
   TupleTableSlot *outerslot = fetch_input_tuple(aggstate);
   initialize_aggregates(aggstate, pergroups);
   for (;;)
   {
        if (aggstate->aggstrategy == AGG_MIXED &&
            aggstate->current_phase == 1)
        {
            lookup_hash_entries(aggstate);
        }
        advance_aggregates(aggstate);
        outerslot = fetch_input_tuple(aggstate);
   }
   finalize_aggregates(aggstate, peragg, pergroups);
}
```

```
static TupleTableSlot *ExecAgg(PlanState *pstate)
{
   AggState    *node = castNode(AggState, pstate);
   TupleTableSlot *result = NULL;
   switch (node->phase->aggstrategy)
   {
      case AGG_HASHED:
         if (!node->table_filled)
             agg_fill_hash_table(node);
         /* FALLTHROUGH */
      case AGG_MIXED:
         result = agg_retrieve_hash_table(node);
         break;
      case AGG_PLAIN:
      case AGG_SORTED:
         result = agg_retrieve_direct(node);
         break;
   }
}
```

```
static TupleTableSlot *ExecAgg(PlanState *pstate)
{
    AggState    *node = castNode(AggState, pstate);
    TupleTableSlot *result = NULL;
    switch (node->phase->aggstrategy)
    {
        case AGG_HASHED:
            if (!node->table_filled)
                agg_fill_hash_table(node);
            /* FALLTHROUGH */
        case AGG_MIXED:
            result = agg_retrieve_hash_table(node);
            break;
        case AGG_PLAIN:
        case AGG_SORTED:
            result = agg_retrieve_direct(node);
            break;
    }
}
```

tantor

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

Спасибо!

www.tantorlabs.ru